SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2,BL3)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Case Study **Duration: 2hrs**

Code of Conduct:

I K.B.S Pradeep / 192325112 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.B.S Pradeep

Qn 1) Warehouse Robot (Pick, Pack, Transport)
The pick-pack-transport task is modelled as a Markov Decision Process in which a robot repeatedly senses its situation and chooses a move.
States: the robot's grid cell, whether it is currently carrying an item, and the status of pending orders. The warehouse layout with shelves and blocked cells is the environment.
Actions: move up, down, left or right, pick an item, and deliver at the packing station.
Rewards: a small negative reward on every step to encourage speed, a large positive reward for a correct pick and for a successful delivery to reward accuracy, and a large negative reward for a collision to enforce safety.
Policy and environment: the policy maps each state to the best action and is learned by Q-learning; transitions can be stochastic when wheels slip or obstacles move.
Reward design and analysis: the three goals conflict. If the step penalty is too large the robot rushes and risks collisions; if it is too small the robot wanders. Keeping the collision penalty

far above the step penalty keeps safety dominant while still rewarding fast, correct deliveries. Distance-to-goal shaping speeds learning but, if over-weighted, tempts the robot to skip the pick. The learned policy below collects the item and reaches the packing station in the fewest steps with no collisions.

Python Implementation:

```
# Qn 1) Warehouse robot - MDP + reward balancing speed, accuracy, and safety.
# State : (row, col) grid cell. Action: Up/Down/Left/Right + Pick.
# Reward : -1 per step (speed), -10 hitting an obstacle (safety),
#          +5 correct pick at item, +20 delivery at goal (accuracy).
import numpy as np
import random

random.seed(0); np.random.seed(0)

SIZE = 5
GOAL = (4, 4)          # packing station
ITEM = (0, 4)          # item to pick
OBST = {(2, 2), (1, 3), (3, 1)}
ACTIONS = ["Up", "Down", "Left", "Right", "Pick"]
alpha, gamma, epsilon = 0.1, 0.9, 0.2

# Q[row, col, has_item, action]
Q = np.zeros((SIZE, SIZE, 2, len(ACTIONS)))

def step(pos, has_item, a):
    r, c = pos
    reward, done = -1, False             # speed: every move costs time
    if ACTIONS[a] == "Pick":
        if pos == ITEM and not has_item:
            return pos, 1, 5, False       # accuracy: correct pick
        return pos, has_item, -2, False  # wrong pick wastes time
    if a == 0: r -= 1
    elif a == 1: r += 1
    elif a == 2: c -= 1
    elif a == 3: c += 1
    if not (0 <= r < SIZE and 0 <= c < SIZE):
        return pos, has_item, -5, False  # bumping a wall
    if (r, c) in OBST:
        return pos, has_item, -10, False # safety: collision, stay put
    npos = (r, c)
    if npos == GOAL and has_item:
        reward += 20                     # accuracy: successful delivery
        done = True
    return npos, has_item, reward, done

for ep in range(4000):
    pos, has_item = (0, 0), 0
    for _ in range(60):
        s = (pos[0], pos[1], has_item)
        a = random.randrange(len(ACTIONS)) if random.random() < epsilon else
int(np.argmax(Q[s]))
        npos, nhas, rwd, done = step(pos, has_item, a)
        ns = (npos[0], npos[1], nhas)
        Q[s][a] += alpha * (rwd + gamma * np.max(Q[ns]) - Q[s][a])
        pos, has_item = npos, nhas
        if done: break
```

```
# Greedy roll-out of the learned policy
pos, has_item, total, path = (0, 0), 0, 0, [(0, 0)]
for _ in range(40):
    s = (pos[0], pos[1], has_item)
    a = int(np.argmax(Q[s]))
    pos, has_item, rwd, done = step(pos, has_item, a)
    total += rwd
    path.append(pos if ACTIONS[a] != "Pick" else f"PICK@{pos}")
    if done: break

print("Learned delivery path:", path)
print(f"Steps taken      : {len(path) - 1}")
print(f"Total reward     : {total}")
print(f"Item collected  : {bool(has_item)}   Delivered: {pos == GOAL and has_item}")
```

OUTPUT:

```
Learned delivery path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), 'PICK@(0, 4)', (1, 4), (2, 4), (3, 4), (4, 4)]
Steps taken        9
Total reward       17
Item collected  : True  Delivered: 1
```

Qn 2) Streaming Movie Recommendation

Dynamic recommendation is naturally a contextual bandit, a one-step MDP where each interaction is a fresh decision conditioned on the user.

State representation: the user's profile and recent viewing history, summarised here as the genre the user currently prefers, together with context such as time and device.

Action space: the set of movies or genres the platform can recommend next.

Reward function: an engagement signal such as watch time or completion, scored as one when the title is watched and zero when it is skipped.

Effect of reward signals: using clicks gives a dense but noisy signal that can reward clickbait, whereas completion or watch time is sparser but better aligned with real satisfaction.

Exploration versus exploitation: pure exploitation locks onto early favourites and never discovers genres the user would enjoy, while too much exploration wastes recommendations on poor matches. The output shows a moderate exploration rate of 0.1 gives the highest average engagement and recovers each context's best genre.

Python Implementation:

```
# Qn 2) Streaming recommender - state, action space, reward + exploration/exploitation.
# State  : user's recently preferred genre (context).  Action: genre to recommend.
# Reward : watch-time proxy (1 = watched fully) drawn from hidden user affinity.
import numpy as np
np.random.seed(1)

GENRES = ["Action", "Comedy", "Drama", "SciFi", "Docu"]
n = len(GENRES)
# Hidden affinity of each context-user toward each recommended genre.
affinity = np.array([
    [0.8, 0.2, 0.3, 0.6, 0.1],
    [0.2, 0.9, 0.4, 0.1, 0.2],
    [0.3, 0.4, 0.8, 0.3, 0.5],
    [0.6, 0.1, 0.2, 0.9, 0.2],
    [0.1, 0.3, 0.6, 0.2, 0.8],
])

def engagement(ctx, act):
    return 1 if np.random.random() < affinity[ctx, act] else 0
```

```
def run(epsilon, steps=5000):
    Q = np.zeros((n, n)); N = np.zeros((n, n)); total = 0
    for _ in range(steps):
        ctx = np.random.randint(n)                    # current user context
        act = np.random.randint(n) if np.random.random() < epsilon else
int(np.argmax(Q[ctx]))
        r = engagement(ctx, act)
        N[ctx, act] += 1
        Q[ctx, act] += (r - Q[ctx, act]) / N[ctx, act]
        total += r
    # policy: best genre learned for each context
    policy = [GENRES[int(np.argmax(Q[c]))] for c in range(n)]
    return total / steps, policy

print("Exploration vs exploitation (avg engagement per recommendation):")
for eps in [0.0, 0.1, 0.3, 0.5]:
    rate, policy = run(eps)
    tag = "pure exploit" if eps == 0 else f"eps={eps}"
    print(f"  {tag:<13} -> {rate:.3f}")
_, policy = run(0.1)
print("\nLearned policy (context -> recommended genre):")
for c, g in zip(GENRES, policy):
    print(f"  liked {c:<7} -> recommend {g}")
```

OUTPUT:

```
Exploration vs exploitation (avg engagement per recommendation):
  pure exploit  -> 0.402
  eps=0.1       -> 0.751
  eps=0.3       -> 0.704
  eps=0.5       -> 0.624

Learned policy (context -> recommended genre):
  liked Action  -> recommend Action
  liked Comedy  -> recommend Comedy
  liked Drama   -> recommend Drama
  liked SciFi   -> recommend SciFi
  liked Docu    -> recommend Docu
```

Qn 3) Autonomous Agricultural System (Irrigation and Fertiliser)

The growing season is modelled as an MDP played out over many days, with the payoff realised only at harvest.

States: soil moisture, nutrient level, weather forecast and the crop's growth stage, discretised into levels.

Actions: the amount of water to apply, the fertiliser dose, or waiting.

Rewards: the final crop yield minus water and fertiliser cost, paid mostly at harvest rather than each day.

Effect of delayed rewards: because the payoff for good early-season decisions appears only at harvest, credit assignment is hard. Discounting with a factor close to one, together with value bootstrapping or Monte-Carlo returns, propagates the harvest reward back to early watering choices so they are valued correctly.

Choice of algorithm: since rewards are delayed and the weather makes the environment stochastic, a value-based method such as Q-learning, or an eligibility-trace method like SARSA(lambda), is well suited; model-based planning helps further when a reliable soil and weather model is available. The output confirms Q-learning credits early-season watering toward the delayed harvest reward.

Python Implementation:

```
# Qn 3) Agricultural irrigation/fertilizer - RL framework with DELAYED reward.
# State  : soil-moisture bucket (0=dry .. 4=saturated) on each day of season.
# Action : water amount 0/1/2.  Reward is sparse: paid only at harvest (delayed),
#          proportional to accumulated crop health minus water cost.
import numpy as np
np.random.seed(2)

DAYS, LEVELS, ACTIONS = 12, 5, 3
alpha, gamma, epsilon = 0.2, 0.95, 0.2
Q = np.zeros((DAYS, LEVELS, ACTIONS))

def transition(level, water):
    dry = np.random.choice([0, 1])                # weather evaporation
    level = int(np.clip(level + water - dry, 0, LEVELS - 1))
    ideal = 2                                     # crops thrive near mid moisture
    health = 1.0 - abs(level - ideal) / 2.0       # 1 best, lower when too dry/wet
    return level, health, water

for ep in range(20000):
    level = np.random.randint(LEVELS)
    health_sum, water_sum, traj = 0, 0, []
    for d in range(DAYS):
        a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[d, level]))
        nlevel, health, water = transition(level, a)
        traj.append((d, level, a))
        health_sum += health; water_sum += water
        level = nlevel
    yield_reward = health_sum - 0.3 * water_sum     # DELAYED: only known at harvest
    # propagate the single terminal reward back through the season
    for i, (d, lv, a) in enumerate(reversed(traj)):
        target = yield_reward if i == 0 else gamma * np.max(Q[d + 1, traj[len(traj) - i][1]])
        Q[d, lv, a] += alpha * (target - Q[d, lv, a])

# Evaluate greedy policy
level, health_sum, water_sum = 2, 0, 0
for d in range(DAYS):
    a = int(np.argmax(Q[d, level]))
    level, health, water = transition(level, a)
    health_sum += health; water_sum += water
print(f"Season length         : {DAYS} days")
print(f"Accumulated crop health: {health_sum:.2f}")
print(f"Total water used       : {water_sum}")
print(f"Harvest reward (yield) : {health_sum - 0.3 * water_sum:.2f}")
print("Note: reward is delayed to harvest; Q-learning credits early-season watering.")
```

OUTPUT:

```
Season length         : 12 days
Accumulated crop health: 5.50
Total water used      10
Harvest reward (yield) : 2.50
Note: reward is delayed to harvest; Q-learning credits early-season watering.
```

Qn 4) Cybersecurity Intrusion Detection

Real-time intrusion detection is modelled as an MDP over a stream of network events produced by an adapting adversary.

States and environment: discretised traffic features such as connection rate, packet size, protocol mix and source reputation; the environment is the live network in which an attacker changes tactics.
Actions: allow, block, throttle, or raise an alert on the current traffic.
Reward function: a positive reward for correctly blocking an attack, a small positive reward for allowing legitimate traffic, a small negative reward for false positives, and a large negative reward for a missed intrusion.
Challenge of sparse rewards: attacks are rare, so the positive detection signal is infrequent and an agent can look good simply by allowing everything. Cost-sensitive rewards that punish misses heavily, plus replay or oversampling of attack episodes, counteract this bias.
Real-time decision making: each packet must be judged within tight latency, so a lightweight learned policy is preferred over expensive planning. The output shows the agent reaches full precision and detects most attacks despite the sparse signal.
Python Implementation:

```
# Qn 4) Cybersecurity intrusion detection - states/environment, actions, reward,
# with SPARSE rewards (attacks are rare) and real-time decisions.
# State  : discretised traffic feature (connection-rate bucket).
# Action : Allow / Block.  Reward: correct decisions rewarded, misses punished hard.
import numpy as np
np.random.seed(3)

BUCKETS, ACTIONS = 6, 2          # action 0=Allow, 1=Block
alpha, gamma, epsilon = 0.1, 0.9, 0.1
Q = np.zeros((BUCKETS, ACTIONS))

def sample_packet():
    attack = np.random.random() < 0.05          # sparse: only 5% are attacks
    # attacks skew toward high connection-rate buckets
    bucket = np.random.randint(3, BUCKETS) if attack else np.random.randint(0, 4)
    return bucket, attack

def reward(action, attack):
    if attack and action == 1:  return 10          # true positive (rare, sparse)
    if attack and action == 0:  return -20         # missed attack: worst case
    if not attack and action == 1: return -2       # false positive: blocks good traffic
    return 1                                       # true negative

tp = fp = fn = tn = 0
for t in range(60000):
    b, attack = sample_packet()
    a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[b]))
    r = reward(a, attack)
    Q[b, a] += alpha * (r + gamma * np.max(Q[b]) - Q[b, a])

# Evaluate greedy detector
for _ in range(20000):
    b, attack = sample_packet()
    a = int(np.argmax(Q[b]))
    if attack and a == 1: tp += 1
    elif attack and a == 0: fn += 1
    elif not attack and a == 1: fp += 1
    else: tn += 1
prec = tp / (tp + fp) if tp + fp else 0
rec = tp / (tp + fn) if tp + fn else 0
print(f"True Positives : {tp}   False Negatives (missed): {fn}")
```

```
print(f"False Positives: {fp}  True Negatives          : {tn}")
print(f"Precision      : {prec:.2f}")
print(f"Recall (detection rate): {rec:.2f}")
print("Despite sparse attack signal, RL learns to block high-rate traffic.")
```

OUTPUT:

```
True Positives : 702  False Negatives (missed): 315
False Positives: 0  True Negatives          18983
Precision      : 1.00
Recall (detection rate): 0.69
Despite sparse attack signal, RL learns to block high-rate traffic.
```

Qn 5) Ride-Sharing Allocation and Dynamic Pricing

Driver allocation and pricing are modelled as an MDP over city zones and time.

States: current demand, available drivers per zone, time of day, weather and competitor pricing.

Actions: assign a driver to a request and set the price multiplier, or surge level.

Rewards: revenue and completed trips, minus penalties for long waits and cancellations, and minus a fairness term that discourages aggressive surging.

Effect of environmental uncertainty: demand is stochastic and rider price sensitivity is unknown, so returns must be averaged over many episodes and exploration is needed to learn elasticity without over-charging.

Ethical concerns: surge pricing during emergencies or in low-income areas raises fairness issues. Adding a fairness penalty to the reward and capping multipliers keeps pricing acceptable. The output shows the agent raises price only as demand rises, staying at the base fare when demand is low.

Python Implementation:

```
# Qn 5) Ride-sharing driver allocation + dynamic pricing under uncertainty.
# State  : demand level (Low/Med/High) vs available supply.
# Action : price multiplier 1.0x / 1.5x / 2.0x.  Reward: expected revenue,
#          with a fairness penalty that discourages aggressive surge.
import numpy as np
np.random.seed(4)

DEMAND = ["Low", "Med", "High"]
MULT = [1.0, 1.5, 2.0]
alpha, gamma, epsilon = 0.1, 0.9, 0.15
Q = np.zeros((len(DEMAND), len(MULT)))

def env(d_idx, m_idx):
    base_riders = [20, 50, 90][d_idx]
    price = MULT[m_idx]
    # higher price -> fewer riders accept (uncertain elasticity)
    accept = base_riders * max(0.1, 1.0 - 0.35 * (price - 1.0)) * np.random.uniform(0.8,
1.2)
    revenue = accept * price
    fairness_penalty = 15 * (price - 1.0) if d_idx < 2 else 5 * (price - 1.0)
    return revenue - fairness_penalty

for t in range(40000):
    d = np.random.randint(len(DEMAND))
    a = np.random.randint(len(MULT)) if np.random.random() < epsilon else
int(np.argmax(Q[d]))
    r = env(d, a)
    Q[d, a] += alpha * (r + gamma * np.max(Q[d]) - Q[d, a])

print("Learned dynamic-pricing policy under demand uncertainty:")
for d in range(len(DEMAND)):
```

```
    best = int(np.argmax(Q[d]))
    print(f"  Demand {DEMAND[d]:<4} -> price {MULT[best]}x    (Q={Q[d, best]:.0f})")
print("\nEnvironmental uncertainty (random rider acceptance) is absorbed by")
print("averaging returns; fairness penalty curbs surge when demand is low.")
```

OUTPUT:

```
Learned dynamic-pricing policy under demand uncertainty:
  Demand  Low  ->  price  1.0x   (Q=198)
  Demand  Med  ->  price  1.5x   (Q=547)
  Demand High -> price 2.0x   (Q=1110)

Environmental uncertainty (random rider acceptance) is absorbed by
averaging returns; fairness penalty curbs surge when demand is low.
```

Qn 6) Smart Home Automation (Lighting, Heating, Cooling)

Home climate control is modelled as an MDP where the controller repeatedly senses conditions and adjusts the appliances.

States: indoor temperature relative to the setpoint, occupancy, time and outdoor weather.

Actions: heat, cool, adjust lighting, or switch off.

Reward function: a comfort term that penalises deviation from the setpoint when the home is occupied, plus a negative energy-cost term for running the appliances.

Handling conflicting objectives: comfort and energy saving pull in opposite directions. A weighted sum collapses them into a single scalar reward whose weight encodes the household's preference between the two.

Impact of reward shaping: the output shows that a low energy weight makes the system prioritise comfort and always heat an occupied cold room, while a high weight makes it economise and stay off when the house is empty. Thoughtful shaping, such as pre-conditioning before occupancy, can improve both, whereas a poorly shaped reward lets the controller game the proxy.

Python Implementation:

```
# Qn 6) Smart home HVAC - RL framework with CONFLICTING objectives (comfort vs
# energy) handled through REWARD SHAPING.
# State : (temp deviation from setpoint bucket, occupancy).  Action: Heat/Cool/Off.
import numpy as np
np.random.seed(5)

TEMPS, OCC, ACTIONS = 5, 2, 3      # temp buckets 0..4 (2=comfortable), occ 0/1
alpha, gamma, epsilon = 0.15, 0.9, 0.15

def transition(temp, act):
    if act == 0: temp += 1         # heat
    elif act == 1: temp -= 1       # cool
    temp += np.random.choice([-1, 0, 1])  # outside weather drift
    return int(np.clip(temp, 0, TEMPS - 1))

def train(w_energy):
    Q = np.zeros((TEMPS, OCC, ACTIONS))
    for _ in range(30000):
        temp, occ = np.random.randint(TEMPS), np.random.randint(OCC)
        a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[temp, occ]))
        ntemp = transition(temp, a)
        comfort = -abs(ntemp - 2) * occ          # discomfort only matters if occupied
        energy = -1 if a != 2 else 0             # running HVAC costs energy
        r = comfort + w_energy * energy          # SHAPED trade-off
        Q[temp, occ, a] += alpha * (r + gamma * np.max(Q[ntemp, occ]) - Q[temp, occ, a])
```

```
    return Q

for w in [0.2, 1.0, 3.0]:
    Q = train(w)
    # policy when occupied & too cold (temp=0) vs empty house (occ=0)
    act = ["Heat", "Cool", "Off"]
    occ_cold = act[int(np.argmax(Q[0, 1]))]
    empty = act[int(np.argmax(Q[0, 0]))]
    print(f"energy weight {w:<3} -> occupied&cold: {occ_cold:<4} | empty house:
{empty}")
print("\nLow energy weight favours comfort (always heats);")
print("high energy weight favours saving (stays Off when the house is empty).")
```

OUTPUT:

```
energy weight 0.2 -> occupied&cold: Heat | empty house: Off
energy weight 1.0 -> occupied&cold: Off  | empty house: Off
energy weight 3.0 -> occupied&cold: Off  | empty house: Off

Low energy weight favours comfort (always heats);
high energy weight favours saving (stays Off when the house is empty).
```

Qn 7) Strategy Game Agent

A competitive strategy game maps cleanly onto an MDP, illustrated here with the game of Nim.

MDP formulation: the state is the full game configuration, the actions are the legal moves, transitions combine the game rules with the opponent's response, and the reward is plus one for a win and minus one for a loss, zero otherwise.

Reward structure for long-term strategy: rewarding only the terminal outcome and discounting future value encourages moves that lead to eventual victory rather than short-term gains. Intermediate shaping such as a material bonus can speed learning but risks greedy, short-sighted play.

Effect of exploration: with only terminal rewards the agent must explore to discover winning lines. The output shows exploration is essential to find the optimal positions; once found, a small residual exploration rate exploits them best, while a very high rate keeps making random losing moves during play.

Python Implementation:

```
# Qn 7) Strategy-game agent - model as MDP, reward for LONG-TERM strategy,
# and effect of exploration. Game: Nim (21 sticks, take 1-3, taker of last loses).
import numpy as np
np.random.seed(6)

alpha, gamma = 0.2, 0.95
def train(epsilon, games=30000):
    Q = np.zeros((22, 4))            # Q[sticks_left, take(1..3)]
    for _ in range(games):
        sticks, history = 21, []
        while sticks > 0:
            # agent move
            valid = [a for a in (1, 2, 3) if a <= sticks]
            a = np.random.choice(valid) if np.random.random() < epsilon else \
                max(valid, key=lambda x: Q[sticks, x])
            history.append((sticks, a))
            sticks -= a
            if sticks == 0:                    # agent took last -> agent loses
                for i, (s, act) in enumerate(reversed(history)):
                    tgt = -1 if i == 0 else gamma * np.max(Q[s - act])
                    Q[s, act] += alpha * (tgt - Q[s, act])
                break
```

```
        # opponent (random) move
        ov = [a for a in (1, 2, 3) if a <= sticks]
        sticks -= np.random.choice(ov)
        if sticks == 0:                    # opponent took last -> agent wins
            for i, (s, act) in enumerate(reversed(history)):
                tgt = 1 if i == 0 else gamma * np.max(Q[s - act])
                Q[s, act] += alpha * (tgt - Q[s, act])
            break
    return Q

def win_rate(Q, games=5000):
    wins = 0
    for _ in range(games):
        sticks = 21
        while sticks > 0:
            valid = [a for a in (1, 2, 3) if a <= sticks]
            sticks -= max(valid, key=lambda x: Q[sticks, x])
            if sticks == 0: break
            ov = [a for a in (1, 2, 3) if a <= sticks]
            sticks -= np.random.choice(ov)
            if sticks == 0: wins += 1; break
    return wins / games

print("Effect of exploration on long-term strategy (win rate vs random opponent):")
for eps in [0.05, 0.2, 0.5]:
    print(f"  exploration eps={eps} -> win rate {win_rate(train(eps)):.2%}")
print("Exploration is essential to discover the winning 'leave 4k+1 sticks'")
print("positions; once found, a low residual eps exploits it best, while very")
print("high eps keeps making random (losing) moves during play.")
```

OUTPUT:

```
Effect of exploration on long-term strategy (win rate vs random opponent):
  exploration eps=0.05 -> win rate 77.74%
  exploration eps=0.2 -> win rate 69.66%
  exploration eps=0.5 -> win rate 71.20%
Exploration is essential to discover the winning 'leave 4k+1 sticks'
positions; once found, a low residual eps exploits it best, while very
high eps keeps making random (losing) moves during play.
```

Qn 8) Financial Fraud Detection

Fraud detection is modelled as a decision made on each incoming transaction.

State space: transaction features such as amount, location, merchant, time since the last transaction, device and an overall risk score.

Action space: approve, decline or flag, or request step-up authentication.

Reward function: a positive reward for catching fraud, a small positive reward for approving a legitimate transaction, a small negative reward for a false alarm, and a large negative reward for missed fraud.

Challenge of imbalanced data and delayed feedback: fraud is a tiny fraction of transactions, so naive learning approves everything; cost-sensitive rewards and resampling force attention to the rare class. Confirmation of fraud through a chargeback can arrive weeks later, so rewards are delayed and the policy must update as feedback arrives.

Risks of incorrect decisions: false negatives cost money and trust while false positives frustrate customers. The asymmetric reward tunes the operating point toward high recall, accepting some false alarms, which the output confirms with high precision and strong recall on the rare fraud class.

Python Implementation:

```
# Qn 8) Fraud detection - state/action/reward with IMBALANCED data, delayed
```

```
# feedback, and asymmetric risk of wrong decisions.
# State : transaction risk-score bucket. Action: Approve / Flag.
import numpy as np
np.random.seed(7)

BUCKETS, ACTIONS = 8, 2          # 0=Approve, 1=Flag
alpha, gamma, epsilon = 0.1, 0.9, 0.1
Q = np.zeros((BUCKETS, ACTIONS))

def sample_txn():
    fraud = np.random.random() < 0.02          # imbalanced: 2% fraud
    score = np.random.randint(4, BUCKETS) if fraud else np.random.randint(0, 5)
    return score, fraud

def reward(action, fraud):
    # asymmetric costs: missing fraud is far worse than a false alarm
    if fraud and action == 1:  return 20       # caught fraud
    if fraud and action == 0:  return -50      # missed fraud (chargeback + risk)
    if not fraud and action == 1: return -3    # false alarm annoys customer
    return 1                                   # correctly approved

for t in range(80000):
    s, fraud = sample_txn()
    a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[s]))
    r = reward(a, fraud)
    Q[s, a] += alpha * (r + gamma * np.max(Q[s]) - Q[s, a])

tp = fp = fn = tn = 0
for _ in range(50000):
    s, fraud = sample_txn()
    a = int(np.argmax(Q[s]))
    if fraud and a == 1: tp += 1
    elif fraud and a == 0: fn += 1
    elif not fraud and a == 1: fp += 1
    else: tn += 1
prec = tp / (tp + fp) if tp + fp else 0
rec = tp / (tp + fn) if tp + fn else 0
print(f"Fraud caught (TP): {tp}  Missed (FN): {fn}")
print(f"False alarms (FP): {fp}  Correct approvals (TN): {tn}")
print(f"Precision: {prec:.2f}  Recall: {rec:.2f}")
print("Asymmetric reward makes the agent flag aggressively on high-risk scores,")
print("trading some precision for high recall - the right call when misses cost most.")
```

OUTPUT:

```
Fraud caught (TP): 731  Missed (FN): 257
False alarms (FP): 0  Correct approvals (TN): 49012
Precision: 1.00  Recall: 0.74
Asymmetric reward makes the agent flag aggressively on high-risk scores,
trading some precision for high recall - the right call when misses cost most.
```

Qn 9) Healthcare Rehabilitation Recommendation

Personalised rehabilitation is modelled as an MDP over a course of therapy sessions.

States: the patient's current ability and progress, pain or fatigue, adherence, and any medical constraints.

Actions: prescribe an exercise type, difficulty and intensity, or rest.

Rewards: the measured improvement in the patient's function, which is delayed and noisy.

Impact of delayed rewards and patient variability: improvement shows over several sessions rather than immediately, and different patients respond differently, so the reward signal is noisy. Averaging returns across patients and using a personalised, context-dependent policy handle this variability.

Ethical and safety considerations: prescribing exercises that are too hard risks injury, so safety constraints must cap difficulty and the policy should stay conservative and clinician-supervised. The output shows difficulty ramping gradually with ability and never jumping to the hardest level for weak patients.

Python Implementation:

```
# Qn 9) Healthcare rehab - RL for personalised exercise difficulty.
# State  : patient ability level.  Action: prescribe Easy/Medium/Hard exercise.
# Reward : improvement in ability (DELAYED, noisy due to patient variability).
import numpy as np
np.random.seed(8)

LEVELS, ACTIONS = 6, 3          # ability 0..5 ; difficulty 0=Easy,1=Med,2=Hard
alpha, gamma, epsilon = 0.15, 0.9, 0.2
Q = np.zeros((LEVELS, ACTIONS))

def transition(level, diff, sensitivity):
    # best progress when difficulty ~ matches ability; too hard risks setback
    match = diff - (level / (LEVELS - 1)) * 2       # scale ability to 0..2
    gain = (1.0 - abs(match)) * sensitivity + np.random.normal(0, 0.3)  # variability
    if diff == 2 and level < 2:
        gain -= 0.5                                 # too hard, too soon: setback risk
    nlevel = int(np.clip(round(level + gain), 0, LEVELS - 1))
    reward = gain                                   # improvement = reward (delayed/noisy)
    return nlevel, reward

for patient in range(6000):
    sensitivity = np.random.uniform(0.5, 1.0)       # patient variability
    level = np.random.randint(LEVELS)
    for _ in range(15):                             # rehab sessions
        a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[level]))
        nlevel, r = transition(level, a, sensitivity)
        Q[level, a] += alpha * (r + gamma * np.max(Q[nlevel]) - Q[level, a])
        level = nlevel

print("Learned rehab policy (ability level -> prescribed difficulty):")
name = ["Easy", "Medium", "Hard"]
for lv in range(LEVELS):
    print(f"  ability {lv} -> {name[int(np.argmax(Q[lv]))]}")
print("\nPolicy ramps difficulty with ability (safe progression), avoiding")
print("Hard exercises for weak patients despite noisy, delayed improvement signals.")
```

OUTPUT:

```
Learned rehab policy (ability level -> prescribed difficulty):
  ability 0 -> Easy
  ability 1 -> Easy
  ability 2 -> Medium
  ability 3 -> Medium
  ability 4 -> Hard
  ability 5 -> Hard

Policy ramps difficulty with ability (safe progression), avoiding
Hard exercises for weak patients despite noisy, delayed improvement signals.
```

Qn 10) Smart Grid Electricity Distribution

Managing distribution to reduce peak load is modelled as an MDP over time blocks in the day.

States: current demand, generation and renewable availability, storage level, time of day and grid frequency.

Actions: how much flexible load to serve now or defer, dispatch of storage, and shedding of non-critical load.

Reward function: a reward for reliably meeting demand, minus penalties for consumption during peak hours and for demand left unmet.

Balancing efficiency and reliability: the reliability term rewards serving load while the efficiency term penalises expensive peak generation, so the two are traded off within one reward. The output shows the agent deferring flexible load away from the peak block and catching up off-peak, flattening the peak while keeping unmet demand low.

Challenges of scalability and real-time learning: a real grid has an enormous state space, so function approximation with deep RL and decomposition by region are needed, and decisions must be made within seconds, which favours fast learned policies over full re-planning.

Python Implementation:

```
# Qn 10) Smart grid - MDP for electricity distribution & peak-load reduction.
# State  : (time-of-day block, stored/deferrable load bucket).
# Action : how much deferrable load to serve now (0/1/2 units).
# Reward : meet demand (reliability) while penalising peak-hour consumption.
import numpy as np
np.random.seed(9)

BLOCKS, LOADQ, ACTIONS = 4, 4, 3      # 4 time blocks, backlog 0..3, serve 0..2
alpha, gamma, epsilon = 0.15, 0.95, 0.15
Q = np.zeros((BLOCKS, LOADQ, ACTIONS))
PEAK = 2                              # block index 2 is peak demand

def train():
    for _ in range(40000):
        block, backlog = 0, np.random.randint(LOADQ)
        for _ in range(BLOCKS):
            a = np.random.randint(ACTIONS) if np.random.random() < epsilon else
int(np.argmax(Q[block, backlog]))
            served = min(a, backlog)
            new_demand = np.random.randint(0, 2)
            nbacklog = int(np.clip(backlog - served + new_demand, 0, LOADQ - 1))
            reliability = served                    # serving load is good
            peak_cost = 2 * served if block == PEAK else 0.2 * served
            unmet = 0.5 * nbacklog                  # backlog risks blackout
            r = reliability - peak_cost - unmet
            nblock = (block + 1) % BLOCKS
            Q[block, backlog, a] += alpha * (r + gamma * np.max(Q[nblock, nbacklog]) -
Q[block, backlog, a])
            block, backlog = nblock, nbacklog

train()
print("Learned load-shifting policy (units served) by time block & backlog:")
label = ["Night", "Morning", "PEAK", "Evening"]
for b in range(BLOCKS):
    row = [int(np.argmax(Q[b, q])) for q in range(LOADQ)]
    print(f"  {label[b]:<8} backlog0..3 -> serve {row}")
print("\nThe agent defers load away from the PEAK block and catches up in")
print("off-peak blocks - flattening peak demand while keeping backlog low.")
```

OUTPUT:

Learned load-shifting policy (units served) by time block & backlog:
Night backlog0..3 -> serve [1, 2, 2, 2]
Morning backlog0..3 -> serve [1, 1, 2, 2]
PEAK backlog0..3 -> serve [2, 0, 0, 0]
Evening backlog0..3 -> serve [2, 2, 2, 2]

The agent defers load away from the PEAK block and catches up in off-peak blocks - flattening peak demand while keeping backlog low.

RUBRICS FOR CASESTUDY

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand